Chapter 1: Code Implementation

In this project, all numerical experiments and simulations are written in C++/C with the support of basic iterative solver and simple parallel data management provided by PETSc. We perform numerical experiments in quasi-uniform mesh as defined in ... We talk about implementation of a coupled FEM-FVM code without losing generality and flexibility of optimization in the future.

It structures as follows that a general implementation of unstructured mesh will be given, and a more specific example of application regarding our logically rectangular mesh will be discussed in details. All application are expressed in two dimension scenario.

1.1 Geometry

The mesh in the code is defined inspired by the classical viewpoint of Boundary Representation (BREP) Baumgart (1975). In two-dimension case, the element is bounded by a set of edges, and the edges are bounded by a set of vertex. The first and most important aspect of creating a FEM or FVM code is indexing the geometric objects. Assuming the the following representation of global geometric object.

$$\begin{aligned} \text{Vertex} \quad & V_i, \quad i \in ID_V, \\ \text{Edge} \quad & E_i, \quad i \in ID_E, \\ \text{Cell} \quad & C_i, \quad i \in ID_C, \end{aligned} \tag{1.1}$$

where ID_V, ID_E and ID_C are countable set containing all the non repeating global index for vertices, edges and faces respectively. We use superscript to label the subset of global index, e.g., $ID_V^{E_i} \subset ID_V$ represents the global index of boundary vertex associated with the edge E_i, which is a subset of global index set of vertex.

At the same time, assuming the following representation of local geometric

object.

$$\begin{aligned} \text{Vertex} \quad & v_i, \quad i \in id_v, \\ \text{Edge} \quad & e_i, \quad i \in id_e, \\ \text{Cell} \quad & c_i, \quad i \in id_c \end{aligned} \tag{1.2}$$

where id_v, id_e and id_c are countable set containing local index for vertices, edges and faces corresponding to a given geometrical object. We use superscript to label the target geometrical object, e.g., $id_v^{E_i} = \{0, 1\}$ represents the local index of boundary vertex associated with the edge E_i.

We connect global index and local index with a one-to-one local to global mapping as

$$f : id \to ID. \tag{1.3}$$

In implementation, we usually don't have to actually define a local to global mapping explicitly. Consider an array(vector) containing global index, the array index is a natural representation for local index set. With the help of array representation, we can reverse to a global to local mapping within each target geometrical objects.

1.1.1 General Unstructured Mesh

We start with a brief analysis of what information should a target cell should see from its perspective. Each polygon cell E_i should have the following links to other geometric objects as

- Adjacent polygons connected by edges;
- Additional adjacent polygons connected with vertices, but not sharing an edge;
- Indicident vertices;
- Edges that bounds this element.

Within the local set, we can relate edges and vertices directly. For convinence, each edge j should has links to:

- Two polygons that sharing this edge;
- Two vertices that bounds this edge.

In FEM and FVM computation, we work with a orientable mesh, where "clockwise" and "countercolckwise" can be defined consistently. Hence, we can use half edge data structure to regulate direction in a unstructured polygon mesh. In figure 1.1, we

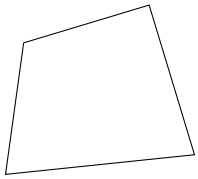

Figure 1.1: An illustration of the half-edge data structure

present an illustration of the half edge data structure applied to a quadrilateral mesh. For each target cell E_i, it sees the vertex V_i and the local link l_i to the next vertex V_i^+. Hence the direction of the link $l_i : V_i \to V_i^+$ is determined from the perspective of the target cell C_i.

1.1.2 Reshape Cartesian Data

The natural of a 2D cartesian data is a pair of index (i, j). However, we usually reshape the 2D data into a 1D array for a more natural representation in computer. We give a more general reshape function that project N-dimensional data to a one-dimensional data. We can also reverse the process reshaping 1D data back into N-dimensional data with modulo and integer division operation.

1.1.3 Logically Rectangular Mesh

Mesh generation is not the focus of this project, hence we adopt a much simpler way of implementing quadrilateral mesh, the logically rectangular way. We generate the quadrialteral mesh by randomly distorting each interior vertex coordinates. There

Algorithm 1 N dimensional reshape

Consider a N-dimensional array $\{n_0 \times n_i \times \dots n_{N-1}\}$.

Let $I = 0$ be the result, and $a = 0$ be the multiplier.

Consider an input N-dimensional index $\{i_0, i_1, \dots, i_{N-1}\}$.

1: **for** rank $r = 0, 1, \dots, N-1$ **do**

2: $\quad I = I + i_r * a.$

3: $\quad a = a \times n_r.$

4: **end for**

is no affine mapping between reference square mesh and generated mesh. The indexing of geometrical objects remain untouched through the vertex distortion.

In our case, all the indices are represented with cartesian coordinates. Therefore, we can setup local to global mapping of geometric objects easily with liear functions. Consider a mesh with $M \times N$ cells, the totlal number of vertex is $(M + 1) \times (N + 1)$ and the total number of edges is $M \times (N + 1) + (M + 1) \times N$. We count horizontal edges first, hence the vertical edges are counted starting from $M \times (N + 1)$. Let local vertex and edge number being counted in the same way as indicated in the mesh **??**, we can define the local to global mapping with local index of vertex and global index of cell. Consider mapping the local index i of vertex v_i in the cell C_I as

$$ID_V \ni = \tag{1.4}$$

and the edge local to global mapping as

1.2 Assembly and dofs

In FEM and FVM computation, we need to associate geometrical objects with degrees of freedom.

1.3 Parallel Implementation

The parallel implementation of the MFEM and

Works Cited

Bruce G. Baumgart. A polyhedron representation for computer vision. In *Proceedings of the May 19-22, 1975, National Computer Conference and Exposition*, AFIPS '75, page 589–596, New York, NY, USA, 1975. Association for Computing Machinery. ISBN 9781450379199. doi: 10.1145/1499949.1500071. URL https://doi.org/10.1145/1499949.1500071.